Name : JETHAVA JAY VINODBHAI

Enrollment no : 250283152004

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import sklearn.preprocessing
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import LabelEncoder

df = pd.read_csv('/content/USA_Housing (1).csv')
df.head()
```

	Avg. Area Income	Avg. Area House Age	Avg. Area Number of Rooms	Avg. Area Number of Bedrooms	Area Population	Price	
0	79545.458574	5.682861	7.009188	4.09	23086.800503	1.059034e+06	208 6
1	79248.642455	6.002900	6.730821	3.09	40173.072174	1.505891e+06	
2	61287.067179	5.865890	8.512727	5.13	36882.159400	1.058988e+06	Strav
3	63345.240046	7.188236	5.586729	3.26	34310.242831	1.260617e+06	US
4	59982.197226	5.040555	7.839388	4.23	26354.109472	6.309435e+05	USN

Next steps: Generate code with df | New interactive sheet

```
df.shape
```

```
(5000, 7)
```

```
df.isnull().sum()
```

	0
Avg. Area Income	0
Avg. Area House Age	0
Avg. Area Number of Rooms	0
Avg. Area Number of Bedrooms	0
Area Population	0
Price	0
Address	0

dtype: int64

No Null values in data sets

```
df.duplicated().sum()
```

```
np.int64(0)
```

No Duplicate values in data sets

```
df.describe() # for Outliers
```

	Avg. Area Income	Avg. Area House Age	Avg. Area Number of Rooms	Avg. Area Number of Bedrooms	Area Population	
count	5000.000000	5000.000000	5000.000000	5000.000000	5000.000000	5.000
mean	68583.108984	5.977222	6.987792	3.981330	36163.516039	1.232
std	10657.991214	0.991456	1.005833	1.234137	9925.650114	3.53
min	17796.631190	2.644304	3.236194	2.000000	172.610686	1.593
25%	61480.562388	5.322283	6.299250	3.140000	29403.928702	9.975
50%	68804.286404	5.970429	7.002902	4.050000	36199.406689	1.232
75%	75783.338666	6.650808	7.665871	4.490000	42861.290769	1.471
max	107701.748378	9.519088	10.759588	6.500000	69621.713378	2.469

No Outliers in our data sets

```
str(df["Address"][1])
```

```
'188 Johnson Views Suite 079\nLake Kathleen, CA 48958'
```

```
str(df["Address"][1]).split('\n')[0]
```

```
'188 Johnson Views Suite 079'
```

```
str(df["Address"][1]).split('\n')[1].split(',')[0]
```

```
'Lake Kathleen'
```

```
str(df["Address"][1]).split('\n')[1].split(',')[1].split()
```

```
['CA', '48958']
```

```
str(df["Address"][1]).split('\n')[1].split(',')[1].split()[1]
```

```
'48958'
```

```
# Split Address into street, city, state and zipcode
def split_address(addr):
    lines = addr.split('\n')
    street = lines[0]
    city, state, zipcode = None, None, None

    if len(lines) > 1:
        second = lines[1]
        if ',' in second:
            city = second.split(',')[0]
            rest = second.split(',')[1].strip().split(maxsplit=1)
            if len(rest) >= 2:
                state, zipcode = rest[0], rest[1]
            elif len(rest) == 1:
                state = rest[0]
        else:
            parts = second.split()
            if len(parts) >= 3:
                city, state, zipcode = parts[0], parts[1], parts[2]

    return pd.Series([street, city, state, zipcode])

df[['Street', 'City', 'State', 'Zipcode']] = df['Address'].apply(split_addre
```

```
df["Street"].unique().shape
```

```
(4982,)
```

```
df["City"].unique().shape
```

```
(3909,)
```

```
df["State"].unique().shape
```

```
(62,)
```

```
df["Zipcode"].unique().shape
```

```
(4958,)
```

```
df.drop(['Address', 'Street'], axis=1, inplace=True)
```

```
df.head()
```

	Avg. Area Income	Avg. Area House Age	Avg. Area Number of Rooms	Avg. Area Number of Bedrooms	Area Population	Price	
0	79545.458574	5.682861	7.009188	4.09	23086.800503	1.059034e+06	Lau
1	79248.642455	6.002900	6.730821	3.09	40173.072174	1.505891e+06	Ka
2	61287.067179	5.865890	8.512727	5.13	36882.159400	1.058988e+06	Dani
3	63345.240046	7.188236	5.586729	3.26	34310.242831	1.260617e+06	
4	59982.197226	5.040555	7.839388	4.23	26354.109472	6.309435e+05	

Next steps: Generate code with df | New interactive sheet

```
le = LabelEncoder()
df["City"] = le.fit_transform(df["City"])
df["State"] = le.fit_transform(df["State"])
df["Zipcode"] = le.fit_transform(df["Zipcode"])
df.head()
```

	Avg. Area Income	Avg. Area House Age	Avg. Area Number of Rooms	Avg. Area Number of Bedrooms	Area Population	Price	City
0	79545.458574	5.682861	7.009188	4.09	23086.800503	1.059034e+06	1610
1	79248.642455	6.002900	6.730821	3.09	40173.072174	1.505891e+06	1476
2	61287.067179	5.865890	8.512727	5.13	36882.159400	1.058988e+06	463
3	63345.240046	7.188236	5.586729	3.26	34310.242831	1.260617e+06	875
4	59982.197226	5.040555	7.839388	4.23	26354.109472	6.309435e+05	875

Next steps: Generate code with df | New interactive sheet

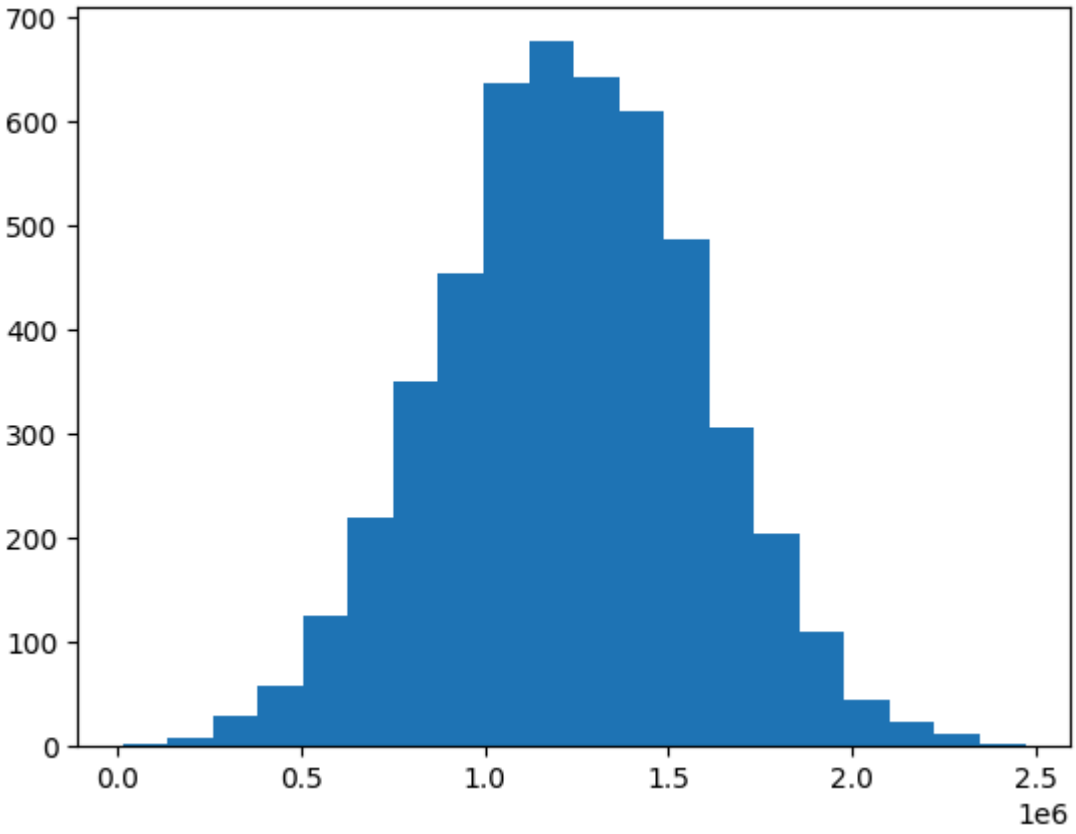
```
cols_to_scale = ['Avg. Area Income', 'Avg. Area House Age', 'Avg. Area Numl

# Normalization
scaler = MinMaxScaler()
df[cols_to_scale] = scaler.fit_transform(df[cols_to_scale])
df.head()
```

	Avg. Area Income	Avg. Area House Age	Avg. Area Number of Rooms	Avg. Area Number of Bedrooms	Area Population	Price	City
0	0.686822	0.441986	0.501502	0.464444	0.329942	1.059034e+06	0.411975
1	0.683521	0.488538	0.464501	0.242222	0.575968	1.505891e+06	0.377687
2	0.483737	0.468609	0.701350	0.695556	0.528582	1.058988e+06	0.118475
3	0.506630	0.660956	0.312430	0.280000	0.491549	1.260617e+06	0.223900
4	0.469223	0.348556	0.611851	0.495556	0.376988	6.309435e+05	0.223900

Next steps: Generate code with df | New interactive sheet

```
plt.hist(df['Price'], bins=20)
plt.show()
```




```
plt.scatter(df['Avg. Area Income'], df['Price'])
plt.xlabel('Avg. Area Income')
plt.ylabel('Price')
plt.title('Avg. Area Income vs Price')
plt.show()
```




```
numeric_data = df.select_dtypes(include=np.number).copy()
arr = numeric_data.corr()

plt.figure(figsize=(10, 8))
sns.heatmap(arr, annot=True, cmap='tab20', fmt='.2f')
plt.title('Correlation Matrix Heatmap')
plt.show()
```



We have 8 Column for Training so Linear Regression function is : Y = β0 + β1X1 + β2X2 + β3X3 + β4X4 + β5X5 + β6X6 + β7X7 + β8X8

where Y = Price X1 = Avg. Area Income X2 = Avg. Area House Age X3 = Avg. Area Number of Rooms X4 = Avg. Area Number of Bedrooms X5 = Area Population X6 = City (after encoding) X7 = State (after encoding) X8 = Zipcode (or encoded Zipcode)

Discrete Mathematics Concepts Used

```
price_scaler = MinMaxScaler()
df['Price'] = price_scaler.fit_transform(df[['Price']])
```

```
# Features
X = df.drop("Price", axis=1).values
```

```
# Target
Y = df["Price"].values.reshape(-1, 1)
```

```
X_train, X_test, Y_train, Y_test = train_test_split(
    X,
```

```
    Y,
    test_size=0.2,
    random_state=42
)
```

```
X.shape
```

```
(5000, 8)
```

```
np.zeros((8, 1))
```

```
array([[0.],
       [0.],
       [0.],
       [0.],
       [0.],
       [0.],
       [0.],
       [0.]])
```

```
# Gradient Descent Function
def gradient_descent(X, Y, learning_rate, epochs):
    m = X.shape[0]
    n = X.shape[1]
    weights = np.zeros((n, 1))
    bias = 0
    loss_history = []

    for epoch in range(epochs):
        # Prediction
        Y_pred = np.dot(X, weights) + bias

        # Error
        error = Y_pred - Y

        # Loss (Mean Squared Error)
        loss = (1/m) * np.sum(error ** 2)
        loss_history.append(loss)

        # Gradient
        dw = (2/m) * np.dot(X.T, error)
        db = (2/m) * np.sum(error)

        # Update
        weights = weights - learning_rate * dw
        bias = bias - learning_rate * db

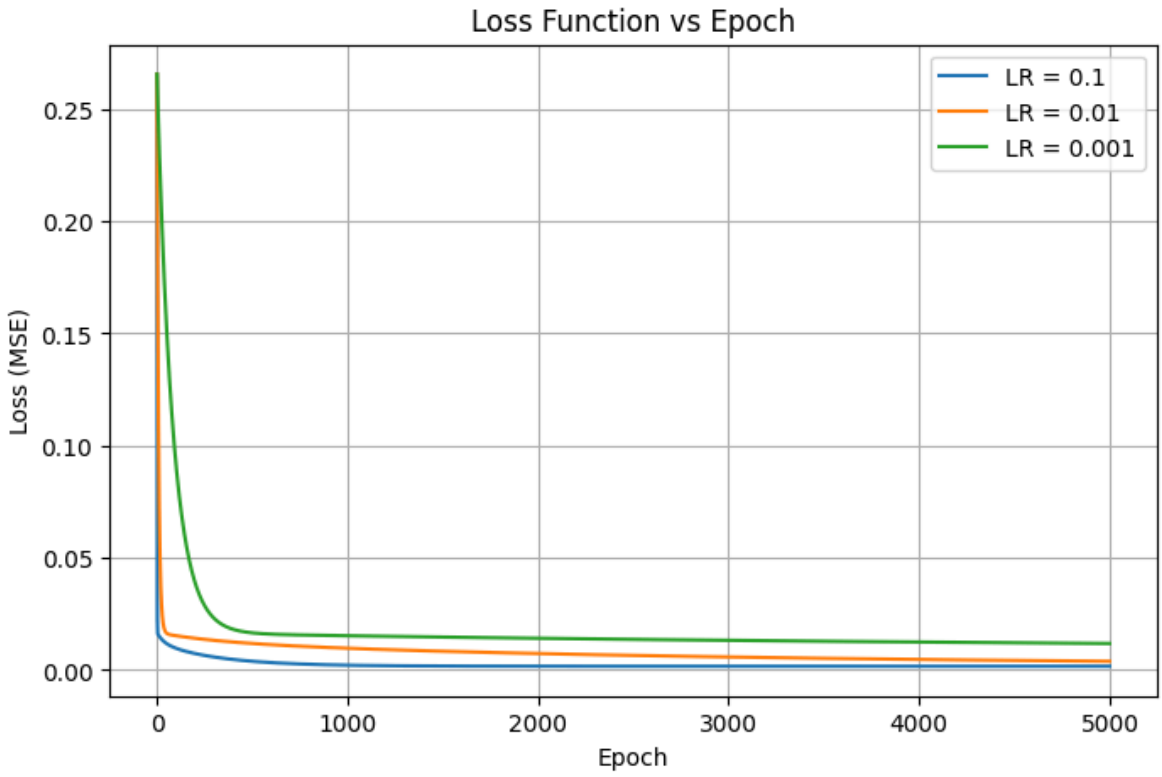
    return weights, bias, loss_history
```

```
learning_rates = [0.1, 0.01, 0.001] # Drastically reduced learning rates
epochs = 5000 # Increased epochs for smaller learning rates
results = {}
```

```
# Training Loop
for lr in learning_rates:
    weights, bias, losses = gradient_descent(
        X_train,
        Y_train,
        lr,
        epochs
    )
    results[lr] = {
        "weights": weights,
        "bias": bias,
        "loss": losses
    }
    print(f"Learning Rate: {lr}")
    print("Final Loss:", losses[-1])
    print("--------------------")
```

```
Learning Rate: 0.1
Final Loss: 0.0017035667269959874
--------------------
Learning Rate: 0.01
Final Loss: 0.0039302292238131776
--------------------
Learning Rate: 0.001
Final Loss: 0.011864404452478106
--------------------
```

```
plt.figure(figsize=(8, 5))
for lr in learning_rates:
    plt.plot(
        results[lr]["loss"],
        label=f"LR = {lr}"
    )
plt.xlabel("Epoch")
plt.ylabel("Loss (MSE)")
plt.title("Loss Function vs Epoch")
plt.legend()
plt.grid(True)
plt.show()
```




```
# Prediction Function
def predict(X, weights, bias):
    return np.dot(X, weights) + bias
```

```
import math

#Test Model
for lr in learning_rates:
    w = results[lr]["weights"]
    b = results[lr]["bias"]

    # Check if weights or bias are NaN
    if np.isnan(w).any() or math.isnan(b):
        print(f"Learning Rate: {lr}")
        print("Model diverged (NaN values in weights/bias). Cannot calculate
        print("--------------------")
        continue # Skip to the next learning rate

    predictions = predict(X_test, w, b)

    mse = mean_squared_error(Y_test, predictions)
    rmse = np.sqrt(mse)
    r2 = r2_score(Y_test, predictions)

    print("Learning Rate:", lr)
    print("MSE:", mse)
    print("RMSE:", rmse)
    print("R2 Score:", r2)
    print("--------------------")
```

```
Learning Rate: 0.1
MSE: 0.001675943955174317
RMSE: 0.04093829448296933
R2 Score: 0.9180252976562371
--------------------
Learning Rate: 0.01
MSE: 0.003870790330264798
RMSE: 0.06221567592066165
R2 Score: 0.810669751706841
--------------------
Learning Rate: 0.001
MSE: 0.01169491044350512
RMSE: 0.10814300922160952
R2 Score: 0.4279720395282657
```

```
# Actual vs Predicted Graph (Best Learning Rate)

# Find a learning rate that produced non-NaN weights and bias
```